Sky Wars

1. System Design and Architecture

Objective

The Sky Wars game is a grid-based game that used Java. The Sky Wars game contain a Sky (technically that is a grid), a Master Spaceship and numbers of Enemy Spaceships like Battle Star, Battle Cruiser and Battle Shooter. The player can play from the point of view of Master Spaceship.

The game is based on 4x4 grid and the player controls Master Spaceship to destroy the enemy spaceships. All the spaceships on the grid move randomly. If the player moves the Master Spaceship, the enemy spaceships also move randomly to the possible square grid that can move to. But the only thing is that enemy spaceships always enter the to the sky (the grid) through the one-directional intergalactic hole in the top left corner of the sky. If there are 2 spaceships in a square of the gird and those spaceships are Master spaceship and one of the enemy spaceships, the Master spaceship will destroy the enemy spaceships and Master spaceship win. If there are 3 spaceships in a square of the grid and those spaceships are Master spaceship and twp of the enemy spaceships, the enemy spaceships will destroy the Master spaceship and the enemy spaceships win. So, this is the whole process of the game.

Class Overview

There are 13 classes in this system.

1. DemoSkyWars
2. SkyWarsGUI
3. Grid
4. GameState
5. BattleShooter
6. BattleStar
7. BattleCruiser
8. MasterSpaceShip

9. Row
10. Square
11. SkyWarsGame
12. EnemySpaceShip
13. Ship

1. DemoSkyWars: This class is to start the game.

2. SkyWarsGUI: This class is the graphical user interface of the game. This class draw the grid and ships, input the user interactions and display the game message from the user interface.

3. Grid: This class represents the game board (sky).

4. GameState: This class is for save and load feature of the game.

5. BattleCruiser: This is one of the enemy ship classes.

6. BattleStar: This is one of the enemy ship classes.

7. BattleCruiser: This is one of the enemy ship classes.

8. MasterSpaceShip: The is ship that is controlled by user.

9. Row: This class represent row of the grid (sky).

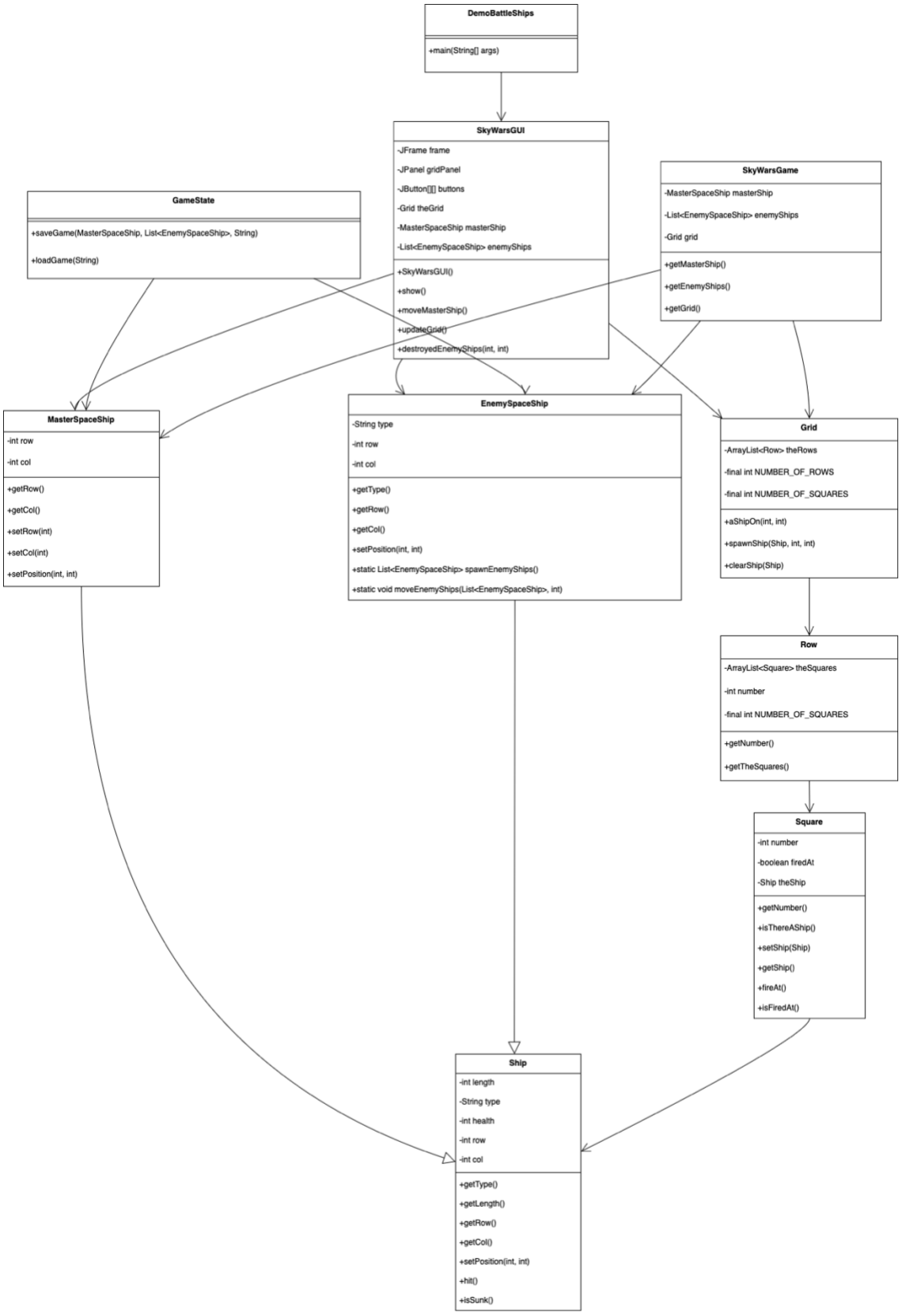
10. Square: This class represent a single tile of the grid.

11. SkyWarsGame: This class is also class for Game Save file in memory.

12. EnemySpaceShip: This class represent the enemy ships and base class of all enemy spaceships.

13. Ship: This class is the base for all roles of ships.

Class Diagram (UML)



2. Polymorphism

In this system, Ship class is used polymorphism and MasterSpaceShip and EnemySpaceShip are inherited from Ship class.

```
public class EnemySpaceShip extends Ship implements Serializable {
```

Fig 2.1: Example of Polymorphism use

This line means that that EnemySpaceShip inherits from the Ship class and can be treated as a Ship type. This enables us to store, display, and apply standard behaviours like location tracking for all ships as if they were of the same kind.

3. GUI Design and Features

Main Feature

- 4x4 Grid
- Ship spawn
- Movement of Spaceships
- Win and lose conditions
- Game State Saving/Loading with serialization

1. 4x4 Grid

The Grid class represent the game grid, and it contain rows and squares. Spaceships can be spawned on this grid.

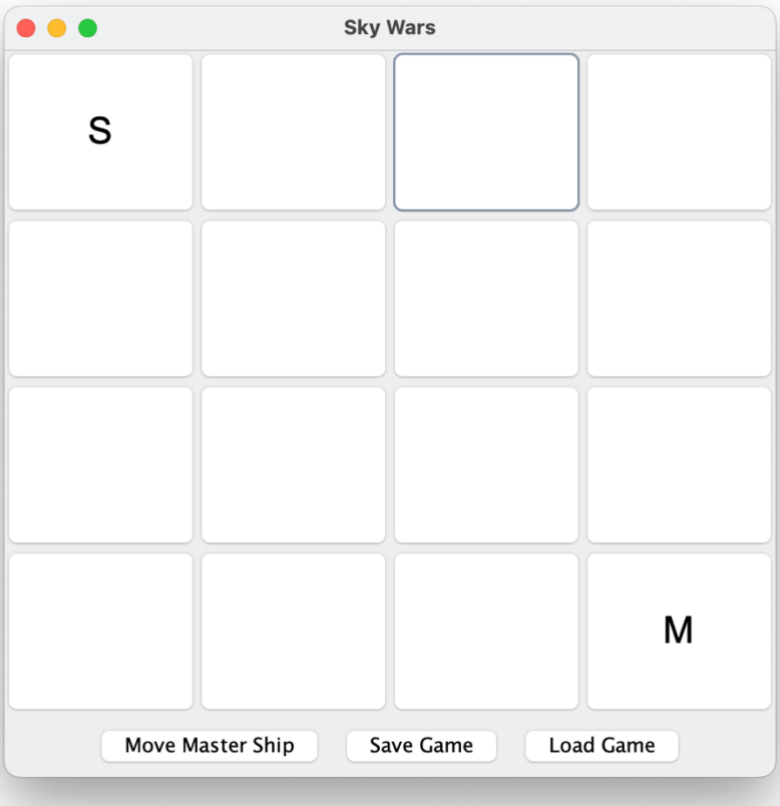


Figure 3.1: The Grid interface

```
package grid01;

import java.util.ArrayList;

public class Grid {

    private ArrayList<Row> theRows;
    private final int NUMBER_OF_ROWS = 4;
    private final int NUMBER_OF_SQUARES = 4;
```

Fig 3.2: Grid class

This means that the number of rows and square are 4 because this is 4x4 grid.

```
public boolean AShipOn(int row, int square) {
    for(Row tempRow : this.theRows) {
        if(tempRow.getNumber() == row) {
            for(Square tempSquare : tempRow.getTheSquares()) {
                if(tempSquare.getNumber() == square) {
                    return tempSquare.isThereAShip();
                }
            }
        }
    }
    return false;
}
```

Fig 3.3: Finding A ship Method

This method is finding a ship at a specific square. It loops through all roles and check each square at that all rows. If it finds a ship on one square, it will return.

2. **Ship spawn**



Fig3.4: Ship Spawn on Grid

All the ship will spawn when the grid appeared. In the interface, Master ship is referred as “M”. Battle star ship is referred as “S”. Battle Shooter ship is referred as “X”. Battle Cruiser ship is referred as “C”.

```
public void spawnShip(Ship ship, int row, int square) {
    for(Row tempRow : this.theRows) {
        if(tempRow.getNumber() == row) {
            for(Square tempSquare : tempRow.getTheSquares()) {
                if(tempSquare.getNumber() == square) {
                    tempSquare.setShip(ship);
                }
            }
        }
    }
}
```

Fig 3.4: Spawn ship method

This method is for ship spawning on grid. It finds the correct row and square and set the ship by using setShip method in Square.

```
public MasterSpaceShip() {
    super("MasterSpaceShip");
    this.row = 3;
    this.col = 3;
}
```

Fig 3.5: Spawn location of Master Spaceship

Master spaceship is spawned in that specific location.

```
public static List<EnemySpaceShip> spawnEnemyShips() {
            enemyShips.clear();

    enemyShips.add(new EnemySpaceShip("BattleStar"));
    enemyShips.add(new EnemySpaceShip("BattleCruiser"));
    enemyShips.add(new EnemySpaceShip("BattleShooter"));

    enemyShips.get(0).setPosition(0, 0);
    enemyShips.get(1).setPosition(0, 0);
    enemyShips.get(2).setPosition(0, 0);

    return enemyShips;
}
```

Fig 3.6: Spawn Enemy Spaceships at Specific location

Enemy spaceships are spawned at Specific location according to the game story line.

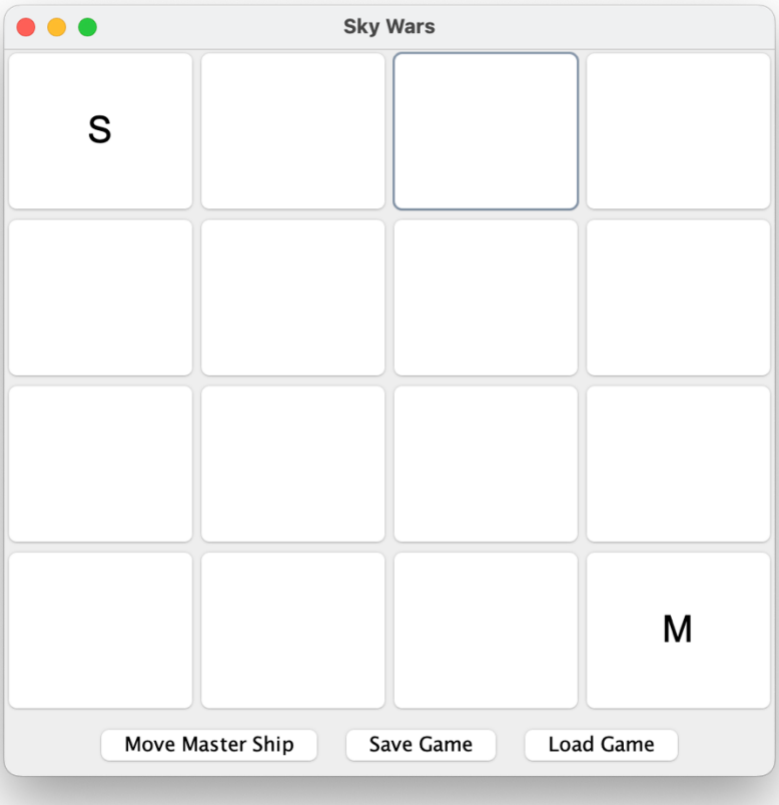


Fig 3.7: Sky Wars Game

In this figure, all the enemy ships are at the top left corner but S is only shown in grid because the system show only one name at the one square but there are 3 ships at that specific location.

3. Movement of Spaceships

All the ships move randomly at the same time when the user move the Master spaceship.



Fig: 3.8: all the spaceships move randomly

Move Master Spaceship

```
// Move the Master SpaceShip
private void moveMasterShip() {

    int row = masterShip.getRow();
    int col = masterShip.getCol();
    int[] dRow = {-1, 1, 0, 0};
    int[] dCol = {0, 0, -1, 1};
    int direction = random.nextInt(4); // gives 0, 1, 2, or 3
    int newRow = row + dRow[direction];
    int newCol = col + dCol[direction];
    if (newRow >= 0 && newRow < 4 && newCol >= 0 && newCol < 4) {

        theGrid.clearShip(masterShip);

        masterShip.setPosition(newRow, newCol);
        theGrid.spawnShip(masterShip, newRow, newCol);
```

Fig 3.9: Move Master Ship Randomly

In this method, currently location of Master spaceships is got and define the possible directions. dRow and dCol means Up (-1,0), Down (1,0), Left (0,-1), Right (0, 1) and choose the random direction by using random method. Then it got new position and check the new possible to move possible or not. After that, clear the old Master spaceship location and set the new location and spawn the Master spaceship.

Move Enemy Spaceships

```
// Move the enemy ship
public void moveRandomly(int gridSize) {
     enemyShips = new ArrayList<>();
    int[] dRow = {-1, 1, 0, 0};
    int[] dCol = {0, 0, -1, 1};
    int direction = (int) (Math.random() * 4);
    int newRow = getRow() + dRow[direction];
    int newCol = getCol() + dCol[direction];

    if (newRow >= 0 && newRow < gridSize && newCol >= 0 && newCol < gridSize) {
        setPosition(newRow, newCol);
    }
}
public static void moveEnemyShips(List<EnemySpaceShip> enemyShips, int gridSize) {

    for (EnemySpaceShip ship : enemyShips) {
        ship.moveRandomly(gridSize); // Move each enemy ship
    }
}
```

Fig 3.10: Move Enemy ships

In this method, it just like moving master ships but here “Math.random” is used unlike moving Master spaceship. After moving method, it loops over each ship and use moveRamdomly(gridSize) to move each enemy ships.

4. Win and lose conditions

Win condition

```
// win condition
 if (enemyShips.isEmpty()) {
     JOptionPane.showMessageDialog(frame, "Congratulations! The Master SpaceShip has destroyed all the enemy shi
     System.exit(0); // End the game
```

Fig: 3.11: Win condition

If there are no enemy ships on the grid, the Master spaceships win.

```
//  Destroys enemy ships
   private void destroyedEnemyShips(int newRow, int newCol) {
       for (EnemySpaceShip ship : enemyShips) {
           if (ship.getRow() == newRow && ship.getCol() == newCol) {
               enemyShips.remove(ship);
               System.out.println("Enemy ship destroyed!");
               JOptionPane.showMessageDialog(frame, "An enemy ship has been destroyed!");

               break;
           }
       }
   }
```

Fig: 3.12: Destroy enemy ships

This method checks that the master ship and one of enemy ships are at the same location or not. If master ship and an enemy ship is same at that location. Master ship destroy the enemy ship.

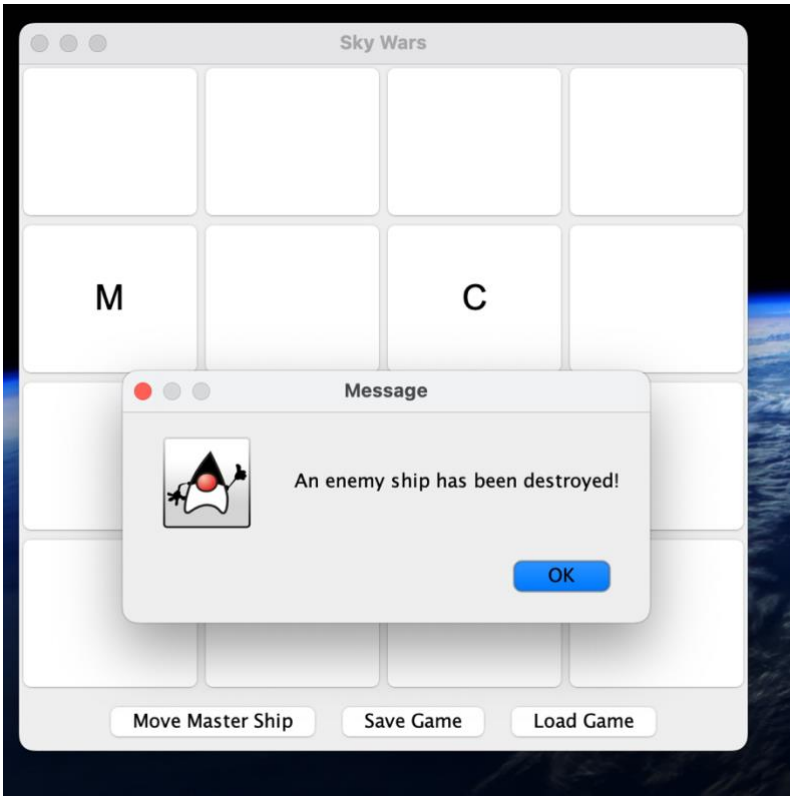


Fig: 3.13: Master spaceship destroyed enemy ship

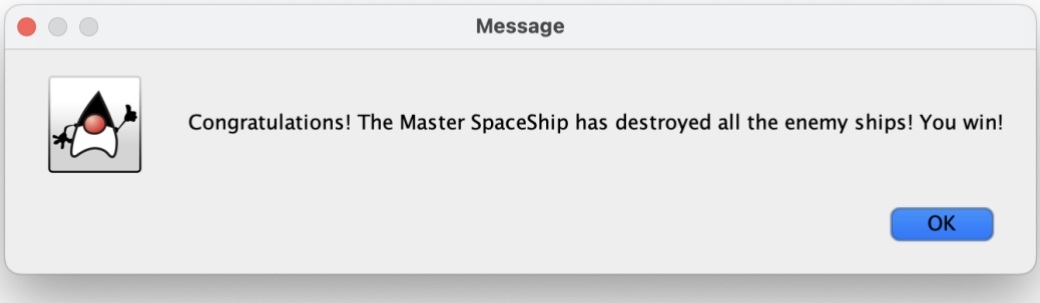


Fig: 3.14: Message box when win

Lose Condition

```
public boolean isGameOver(int newRow, int newCol) {
    int enemyCount = 0;
    for (EnemySpaceShip ship : enemyShips) {
        if (ship.getRow() == newRow && ship.getCol() == newCol) {
            enemyCount++;
        }
    }
    return enemyCount >= 2;
}
```

Fig: 3.15: Game Over method

This method means if the 2 or more enemy ships are same position with Master ships, the enemy destroy the Master ship and game over.

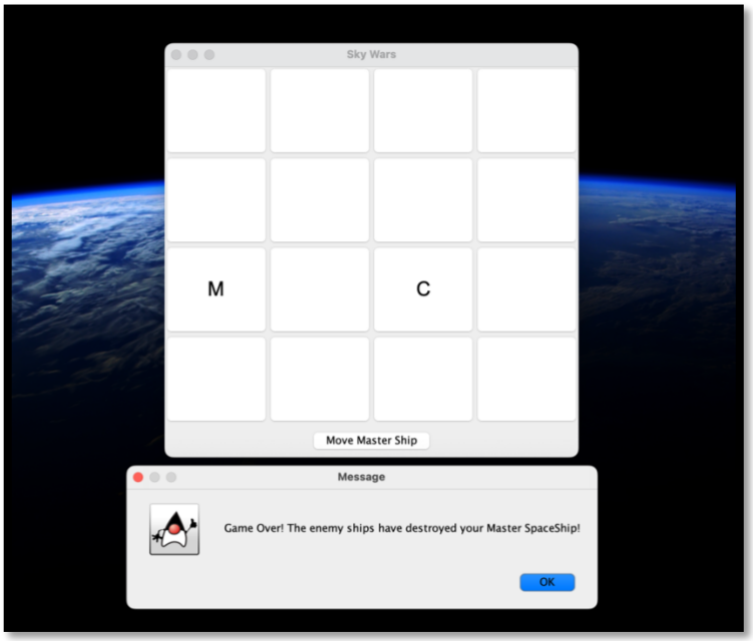


Fig: 3.16: Game Over moment

In this figure, it seems like battle star ship and battle shooter ships are same position with Master spaceship. So, that is the game over.

5. Game State Saving/Loading with serialization

In this game, saving state save the data which is the current player's data to a file so the game can be reused later. Loading state means that stored data are loaded and restoring to the previous state. For the saving and loading state, Serialization is used to convert an object into a byte stream.

```
public static void saveGame(MasterSpaceShip masterShip, List<EnemySpaceShip> enemyShips, String filename) {
    try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(filename))) {
        out.writeObject(masterShip);
        out.writeObject(enemyShips);
        System.out.println("Game saved successfully.");
    } catch (IOException e) {
        e.printStackTrace();
        System.out.println("Failed to save game.");
    }
}
```

Fig 3.17: Saving state

In this method, Master spaceships and Enemy spaceships are saved into a file.

```
public static Object loadGame(String filename) {
    try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(filename))) {
        MasterSpaceShip masterShip = (MasterSpaceShip) in.readObject();
        List<EnemySpaceShip> enemyShips = (List<EnemySpaceShip>) in.readObject();
        System.out.println("Game loaded successfully.");
        return new Object[] { masterShip, enemyShips };
    } catch (IOException | ClassNotFoundException e) {
        e.printStackTrace();
        System.out.println("Failed to load game.");
        return null;
    }
}
```

Fig: 3.18: Load state

This method read the data from a saved files and return the Master ship and Enemy ships to pervious location.

```
public class MasterSpaceShip  extends Ship implements Serializable{
```

Fig: 3.19: implements Serializable

To use Serializable, all the ships that need to be saved must implement Serializable.

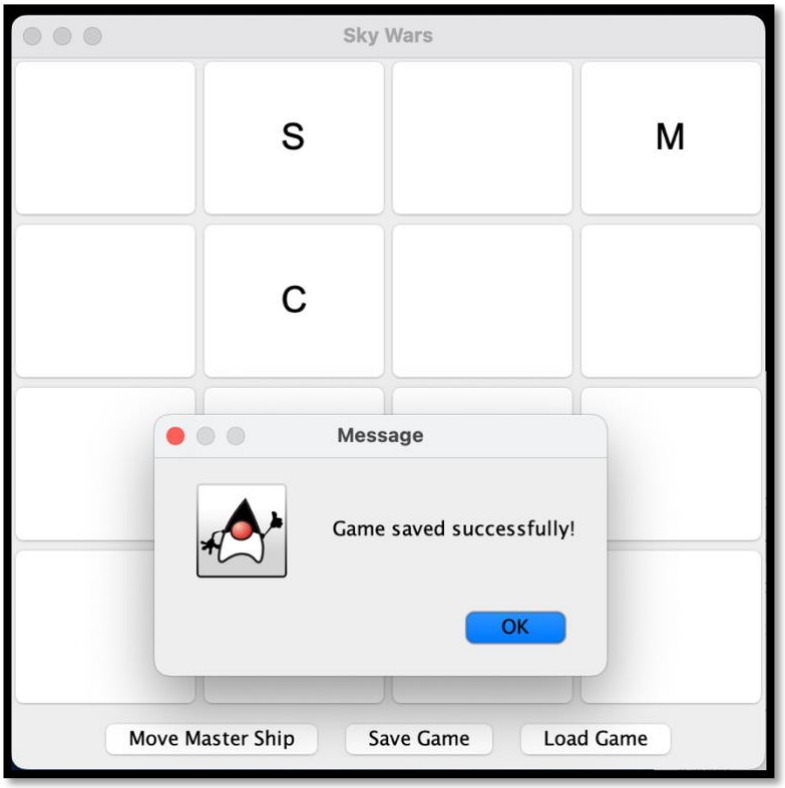


Figure: 3.20: Saved game

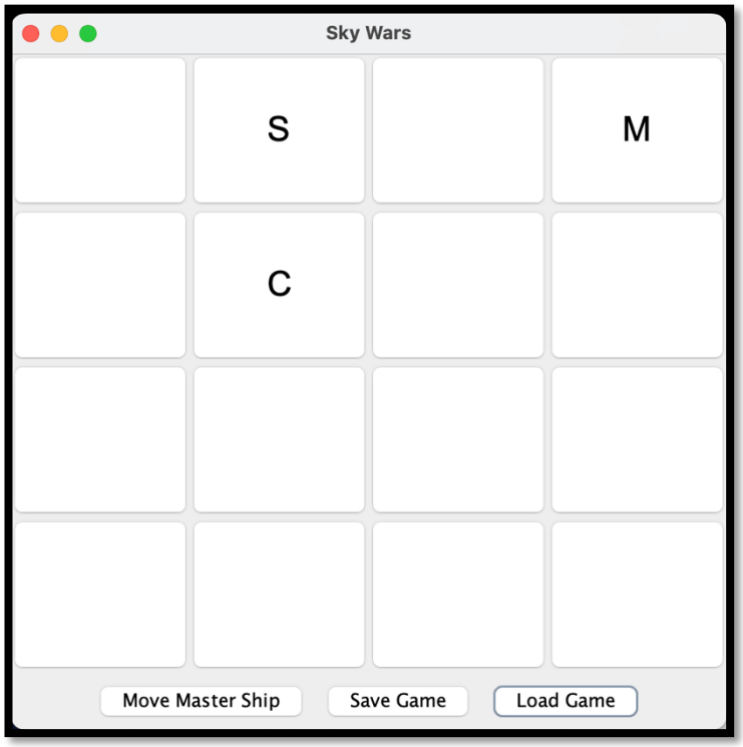


Fig: 3.21: Load Game

4. JUnit Testing

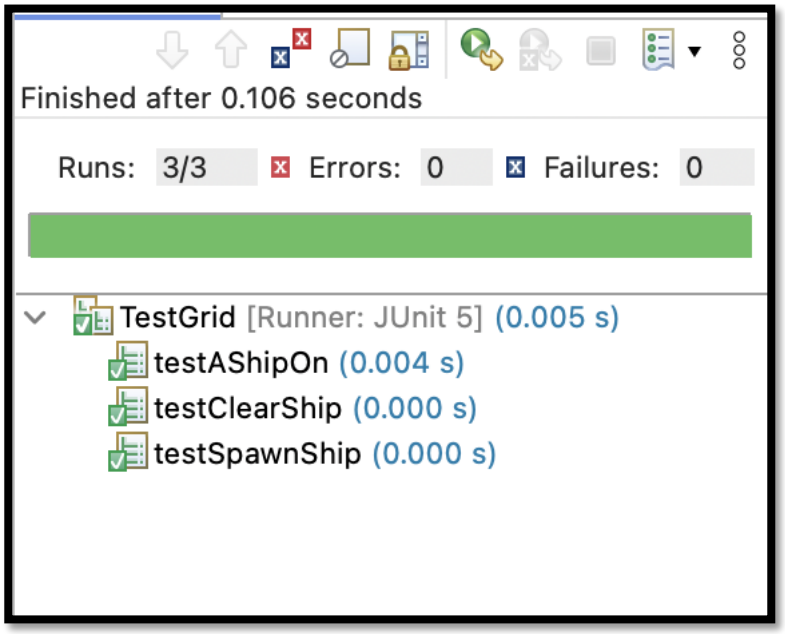


Fig 4.1: JUnit testing for Grid

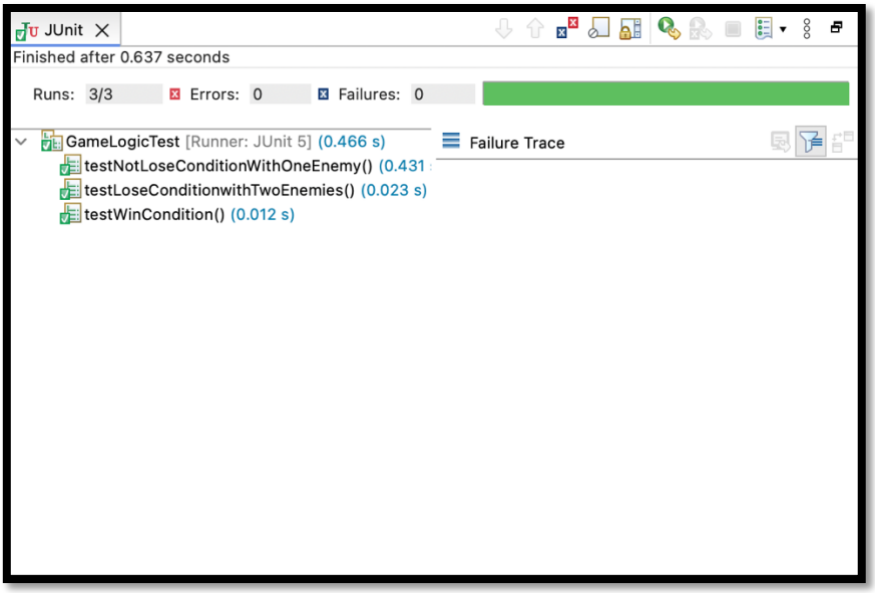


Fig 4.2: JUnit Testing for Win and lost condition

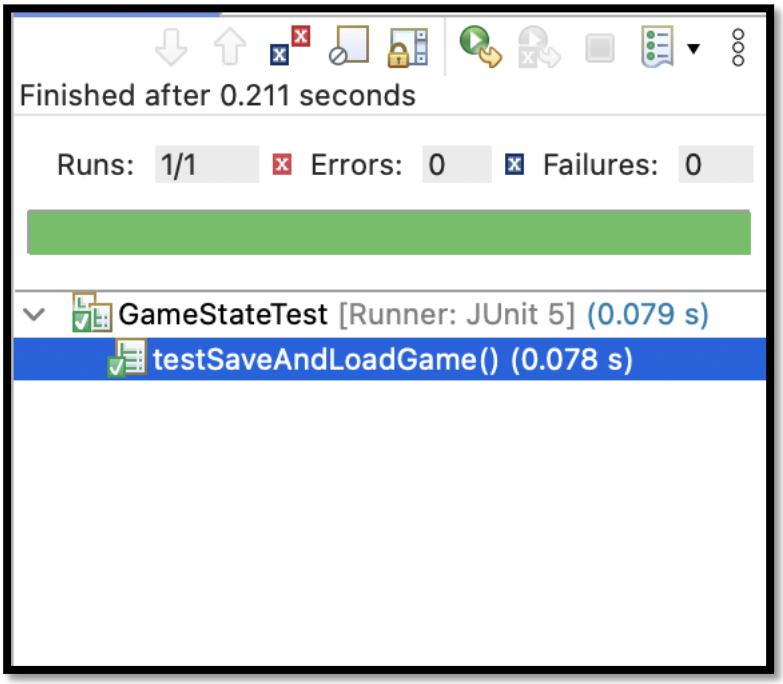


Fig 4.3: Junit Testing for Save and load game state

5. Feature Development and opinions

As technically, this game is not like a real game because there is only one button and that can only move random so the player can't control the ship and also the winning condition is very high because the possible way to place 3 ships at the same time is very rare. So, it like the player always winning. Also, the player also wins when there is only 1 enemy ship because if there is only one enemy ship, there is no way to place two enemy at the same time with Master ship. So, I want to add some feature like the player can chose the position of the Master ship moving or skip the round. I think that would be better and seem like more realistic than this.